

Development Rules & Military Coding Guidelines

Project: Project Rudra-Logistics

Compliance Standard: Defence Edge Applications & Tactical C2 Systems

Audience: All Core Developers & AI Agents

1. Core Principles & Philosophy

1. **Air-Gapped First (Zero External Network Leaks):**

- No external CDNs, fonts, or tracking scripts allowed in frontend templates.
- All map vector layers and assets must be local (GeoJSON, vector MBTiles, or bundled SVG icons).
- Backend must function 100% offline without relying on external third-party cloud APIs.

2. **Deterministic & Mission-Critical Reliability:**

- Supply calculation errors can risk soldiers' lives in a real tactical scenario.
- All math formulas (especially Days of Supply `DOS` and multiplier logic) must have comprehensive unit tests and zero floating-point rounding errors on unit conversions.

3. **Operational Ergonomics (Low Cognitive Load):**

- The user is often an operating officer working under high fatigue and low light.
 - Do not display confusing clutter. Highlights must be actionable: **Red (Immediate action required)**, **Amber (Upcoming issue)**, **Green (Within safe operational limits)**.
-

2. Standard Military Terminology (Domain Lexicon)

Always use authentic military terminology in code variables, database fields, and UI labels:

Military Acronym / Term	Definition in System
DOS	Days of Supplies: Number of days current stock will last based on forecasted consumption.
FOP	Forward Operating Post: Outpost deployed at the line of contact (e.g., Siachen Base Camp).
FLD / ABD	Forward Logistics Depot / Advance Base Depot: Intermediate staging depot (e.g., Partapur).
POL	Petroleum, Oil, and Lubricants: Liquid energy supplies (Kerosene, Diesel, Jet Fuel).
Bukhari	Traditional kerosene/wood-burning space heater used in extreme high altitude.
ALS	All-Terrain Light Special Vehicle: 4x4 troop/cargo carrier capable of steep gradient traversal.
Stallion	Heavy military cargo truck (Ashok Leyland 4x4 / 6x6) used for main highway convoys.
ALH	Advanced Light Helicopter (Dhruv): Tactical helo used for critical high-altitude airdrops.
COP	Common Operating Picture: The single unified command screen showing all operational assets.

3. Backend (Python / FastAPI) Standards

- **Language:** Python 3.10+
- **Code Style:** PEP 8 compliance. Use `black` (line length: 100) and `flake8` for linting.
- **Type Annotations:** All functions and Pydantic models must have strict type hints:

```
python def calculate_dos(current_stock: float, daily_burn_rate: float) → float: if
daily_burn_rate ≤ 0: return 999.0 # Infinite days if consumption is zero return
round(current_stock / daily_burn_rate, 1)
```
- **Async Where Appropriate:** Use `async def` for I/O bound endpoints (database reads, WebSocket broadcasts), and synchronous worker threads for CPU-heavy AI route optimization algorithms.
- **Error Handling:** Standardized JSON error response format:

```
json { "status": "error", "error_code": "PASS_UNREACHABLE", "message": "Both primary and
secondary routes blocked. Helicopter/Drone dispatch mandatory." }
```

4. Frontend (React / Tailwind) Standards

- **Component Organization:** All components must be modular, single-responsibility functional components with Hooks.
 - **Styling:** TailwindCSS exclusively. No inline styles except for dynamic canvas/SVG positions.
 - **Tactical Theme Enforcement:**
 - Backgrounds: `bg-slate-950` or `bg-slate-900` .
 - Borders: `border-slate-800` or `border-zinc-700` .
 - Primary text: `text-slate-100` .
 - Secondary text / telemetry: `text-slate-400` with `font-mono` .
 - Status colors:
 - Critical: `text-rose-500` / `bg-rose-950/60` / `border-rose-600`
 - Warning: `text-amber-400` / `bg-amber-950/60` / `border-amber-600`
 - Nominal: `text-emerald-400` / `bg-emerald-950/60` / `border-emerald-600`
 - **Telemetry HUD:** All numbers (speed, temperature, coordinates, DOS) must use Monospace font (`font-mono`) to prevent visual jitter during live updates.
-

5. Git Commit & Branching Conventions

- Commit messages must follow Conventional Commits:
 - `feat(forecast): add temperature-based kerosene demand multiplier`
 - `feat(gis): add landslide detour calculation for Khardung La`
 - `fix(telemetry): correct cold-chain threshold alarm trigger`
 - `docs(prd): update target corridor to Leh-Siachen`
 - Feature branches: `feat/ai-demand` , `feat/gis-routing` , `feat/frontend-c2-map` .
-

6. Military Cyber Security & Anti-Hacking Coding Standards

- **Zero Hardcoded Secrets:** Never hardcode coordinates of actual classified military installations or test API tokens in git repositories.
- **Strict Input Validation & Sanitization:** All incoming requests MUST pass through Pydantic v2 schemas. Never accept raw unvalidated dictionary parameters.
- **Parameterized SQL Only:** Raw string concatenation in SQL queries (`f"SELECT * FROM items WHERE name = '{var}'"`) is strictly grounds for pull-request rejection. Use SQLAlchemy ORM or

parameterized queries to completely eliminate SQL Injection.

- **Rate-Limiting & Anti-Brute Force:** The `/api/v1/auth/login` and admin mutation endpoints must enforce a strict sliding window limiter (maximum 5 attempts per 60 seconds) with IP/node throttling.
- **Cryptographic Audit Hash Rule:** Every write operation in the admin panel or inventory system MUST compute and append a SHA-256 hash chaining back to the previous entry before committing to the database.
- **Fail-Secure Defaults:** If an anomaly or hash mismatch is detected, the system must fail closed (log the anomaly, alert the Commander, and freeze mutation privileges on that specific record until reviewed).
- **Environment Isolation:** Use `.env.example` file and load configuration via `pydantic-settings`. In mock datasets, use authentic terrain features (Leh, Khardung La, Nubra Valley) with simulated troop strengths and fictive convoy IDs.